

RAGEN：从多轮强化学习视角理解 LLM 智能体的自我进化

——一份面向 AI 应用开发者的中文精读导读

Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang 等
Northwestern University (西北大学) / University of Washington (华盛顿大学) / Stanford University (斯坦福大学)
Microsoft (微软) / New York University (纽约大学) / University of British Columbia / Singapore Management University

英文原题: RAGEN: Understanding Self-Evolution in LLM Agents via Multi-Turn Reinforcement Learning
发表: arXiv 2025 (arXiv:2504.20073) **引用数:** 229 (Semantic Scholar, 2026-06-07)
代码/环境: <https://github.com/RAGEN-AI/RAGEN> **项目主页:** <https://ragen-ai.github.io/>
一句话定位: 这不是又一个刷榜的新方法, 而是认真研究”多轮 agentic RL (智能体强化学习) 为什么训着训着就崩”, 并给出的一套**稳定训练配方**。作者刻意在极简、可控的玩具环境里训练, 把训练动力学 (training dynamics) 看个清楚。**想自己动手训 agent 的人, 避坑必读。**

【阅读说明】本文是面向 AI 应用开发实习生/初学者的”精读导读”, 属于转化性学习材料: 以讲解、转述、关键句翻译为主, 并非对原文的逐字翻译。术语首次出现时括注英文原文; 模型名、数据集名、公式、引用标记按规范保留原样。

1 小白知识卡：读这篇前要懂的背景

这篇论文站在”强化学习 + LLM 智能体”的交叉口上, 下面这些概念先有个大白话印象, 后面读起来会顺很多。

强化学习 (RL, reinforcement learning): 让模型像玩游戏一样, 通过”试一下、看奖励、再调整”来学习。核心三要素是**状态 (state)**、**动作 (action)**、**奖励 (reward)**。

单轮 RL vs. 多轮 RL (single-turn vs. multi-turn): 单轮就是”给一道数学题、答一次、对错给分”, 一问一答就结束 (如解数学题、写代码)。多轮是”模型要连续做好几步决策, 每步环境都会反馈、状态会变” (如推箱子、网购、操作 GUI)。本文的核心就是研究**多轮**场景。

LLM 智能体 (LLM agent): 把大语言模型当成一个能在环境里**自主行动的**”代理人”, 它要做序列决策、跨轮记忆、并适应带随机性的环境反馈。

PPO (Proximal Policy Optimization, 近端策略优化): 最经典的 RL 算法之一。它额外训练一个 **critic (评论家/价值网络)** 来估计”当前局面值多少分”，并用”裁剪 (clipping)”限制每步更新幅度，让训练更稳。

GRPO (Group Relative Policy Optimization, 组相对策略优化): DeepSeek 提出的算法, **不用 critic**。它对同一道题采样一组答案, 用”这组答案里你比平均水平好多少”来当优势 (advantage)。更省, 但少了 critic 的稳定作用。

RLVR (基于可验证奖励的强化学习, RL with Verifiable Reward) / 规则奖励 (rule-based reward): 奖励不靠人打分、也不靠奖励模型, 而是用**规则**自动判定对错 (如数学答案是否正确、箱子是否推到位)。本文用的就是这种规则奖励。

rollout (轨迹采样/推演): 让当前模型在环境里实际跑一遍, 生成一条完整的”状态-动作-奖励”轨迹 (trajectory)。这些自己跑出来的轨迹, 就是 RL 训练的”教材”。

credit assignment (信用分配): 一条多轮轨迹最后成功/失败了, 到底是中间哪一步的功劳或锅? 把最终奖励合理地”分摊”到每一步上, 是多轮 RL 的核心难题。

熵 (entropy) 与奖励方差 (reward variance): 熵衡量模型输出有多”随机/多样”; 奖励方差衡量同一道题的多次尝试结果有多”参差”。两者都高, 说明模型还在积极探索; 两者一塌缩, 往往就是”学僵了”的信号——这正是本文诊断训练崩溃的关键仪表盘。

2 摘要全译

把大语言模型 (LLM) 训练成交互式智能体面临独特挑战, 包括长时程决策 (long-horizon decision making) 以及带随机性的环境反馈进行交互。尽管强化学习 (RL) 已在静态任务上取得进展, 多轮智能体 RL 训练仍未被充分探索。我们提出 **StarPO** (State-Thinking-Actions-Reward Policy Optimization, 状态-思考-动作-奖励策略优化), 一个面向轨迹级 (trajectory-level) 智能体 RL 的通用框架; 并引入 **RAGEN**, 一个用于训练和评估 LLM 智能体的模块化系统。我们在四个风格化 (stylized) 环境上的研究揭示了三个核心发现。第一, 我们的智能体 RL 训练表现出一种反复出现的失效模式——**Echo Trap (回声陷阱)**, 其特征是奖励变化骤降 (cliff) 与梯度尖峰 (gradient spike); 我们用 **StarPO-S** (稳定版变体) 来应对它, 包含轨迹过滤 (trajectory filtering)、引入 critic、以及梯度稳定化。第二, 我们发现 RL rollout 的塑造会从以下方面受益: 多样的初始状态、适中的交互粒度 (interaction granularity)、以及更频繁的采样。第三, 我们表明, 如果没有细粒度、感知推理 (reasoning-aware) 的奖励信号, 智能体的推理能力很难通过多轮 RL 涌现出来, 模型可能表现出肤浅的策略或幻觉式的思考 (hallucinated thoughts)。

关键词: LLM Agents, Multi-turn RL **主页**: <https://ragen-ai.github.io/> **代码/环境**: <https://github.com/RAGEN-AI/RAGEN>

3 逐节精读

3.1 第 1 节：引言——核心问题是什么

一句话讲清：把 LLM 训成能在交互环境里”自我进化”的智能体，最大的拦路虎是**训练不稳定**。这篇就是系统研究”到底什么设计因素能让自我进化的 LLM 智能体学得又好又稳”。

作者先点出多轮 agent 和静态任务（如单轮数学题、写代码）的本质差别：agent 要**连续做决策、跨轮维持记忆、适应随机反馈**。这类场景（规划助手、机器人、辅导 agent）不仅要求模型”表现好”，更要求它能**从经验中自我提升**。

原文一句关键设问值得记住：

”A key open question is: what design factors make self-evolving LLM agents learn effectively and stably?”

（一个关键的开放问题是：什么样的设计因素能让自我进化的 LLM 智能体学得既有效又稳定？）

为回答这个问题，作者搭了两样东西：**StarPO**（一个把”多轮、轨迹级”训练统一起来的 RL 框架，可灵活控制推理、奖励分配、prompt-rollout 结构）和 **RAGEN**（在 StarPO 之上实现完整训练闭环的模块化系统，专门用来做训练动力学的系统性分析）。

评测放在四个复杂度递增的环境上：**Bandit**（单轮、随机）、**Sokoban**（多轮、确定性）、**Frozen Lake**（多轮、随机）、**WebShop**（多轮、开放域）。前三个是极简、完全可控的符号环境，方便”看清”动力学；WebShop 加入了真实世界的语言理解和推理。

引言最后用三条”发现”概括全文（这也是整篇的骨架，后面会逐一展开）：

发现 1（梯度稳定是稳定训练的关键）：多轮 RL 常陷入一种反复出现的不稳定模式 **Echo Trap**（回声陷阱）——智能体过拟合到局部被奖励的推理套路，表现为奖励方差塌缩、熵下降、梯度尖峰。对策是 **StarPO-S**：基于方差的轨迹过滤、critic 基线、以及解耦裁剪（decoupled clipping）。

发现 2（rollout 的频率与多样性塑造自我进化）：LLM 自己生成的轨迹是核心训练材料。关键的 rollout 因素有三个：（1）rollout 要来自**多样的初始状态**，且每个初始状态有多条回应；（2）每轮允许**多个动作**，在固定轮数内拉长交互时程；（3）保持高 **rollout 频率**，让在线反馈反映当前策略。

发现 3（推理的涌现需要精心设计的奖励信号）：仅仅在输出格式上”鼓励推理”并不能保证模型真的去推理。即使用 `<think>` 标签提示、并用 StarPO 做轨迹级优化，如果推理带不来额外奖励优势，模型就会退化回”直接选动作”。而当奖励只反映任务是否成功时，模型会产生**幻觉式推理**——想的和环境状态对不上。这说明长时程 agent 训练需要**细粒度、感知推理**的奖励设计。

3.2 第 2 节：框架——StarPO 是怎么定义的

3.2.1 2.1 把 agent 训练写成 MDP

以前给语言模型做 RL 通常是**单轮**设定：在数据集 $\mathcal{D}$ 上采样 prompt-response 对 (s, a) ，目标是最大化期望奖励：

$$J_{\text{step}}(\theta) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\theta}(\cdot | s)} [R(s, a)]. \quad (1)$$

逐符号拆解 (式 1)： θ 是模型参数； π_{θ} 是**策略** (policy，就是当前模型)； s 是输入 prompt (状态)； a 是模型采样出的回应 (动作)； $R(s, a)$ 是奖励函数； $\mathbb{E}[\cdot]$ 表示“求平均/期望”。整条式子的意思是：让模型在所有 *prompt* 上、按自己的策略采样回应，平均奖励越高越好。

但 LLM 智能体得在**多轮、带随机反馈**的环境里干活。作者把它建模成马尔可夫决策过程 (MDP, Markov Decision Process) $\mathcal{M} = \{S, A, P\}$ ： S 是状态 (如观测序列或交互历史)， A 是动作 (通常是 token 序列)， P 是转移动力学与奖励生成过程。在每个时间步 t ，策略基于当前状态和历史生成动作，环境再返回奖励和新状态：

$$a_t \sim \pi_{\theta}(\cdot | s_t, \tau_{<t}), \quad (r_t, s_{t+1}) \sim P(\cdot | s_t, a_t).$$

其中 $\tau_{<t} = \{s_0, a_0, r_0, \dots, s_{t-1}, a_{t-1}, r_{t-1}\}$ 是**交互历史**。这个过程持续到最大时程 K ，产出一条完整轨迹 $\tau = \{s_0, a_0, r_0, \dots, s_K\}$ ，它就是智能体学习的“教材”。

3.2.2 2.2 StarPO：用轨迹级优化来强化推理

核心思想 (最重要的一段)：和以前“把每个动作独立对待”的静态任务方法不同，StarPO 把**整条轨迹**——包括观测、推理痕迹、动作、反馈——当作一个**连贯整体**来 rollout 和优化。目标是最大化期望轨迹奖励：

$$J_{\text{StarPO}}(\theta) = \mathbb{E}_{\mathcal{M}, \tau \sim \pi_{\theta}} [R(\tau)]. \quad (2)$$

逐符号拆解 (式 2)： $\mathcal{M}$ 是那个 MDP； τ 是一条“被推理增强过”的完整交互序列； $R(\tau)$ 是**整条轨迹**的累积奖励。轨迹概率 $\pi_{\theta}(\tau)$ 可以分解成 token 级的似然，所以 StarPO 天然兼容自回归 LLM。

和老方法的对比一眼看懂：

老方法 (PPO、GRPO)： $J_{\text{step}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot | x)} [R(x, y)]$ ——给输入 x ，优化**单轮输出** y 。

StarPO (本文)： $J_{\text{StarPO}}(\theta) = \mathbb{E}_{\mathcal{M}, \tau \sim \pi_{\theta}} [R(\tau)]$ ——优化**整条轨迹** $\tau = \{s_0, a_0, r_0, \dots, s_K\}$ 上的总奖励。

优化流程：每轮训练，智能体从初始状态 s_0 出发，生成 N 条轨迹。每一步产生一个**被推理引导**的结构化输出：

$$a_t^T = \text{<think>...</think><answer>} a_t \text{</answer>}, \quad (3)$$

这里 a_t^T 是包含中间推理的完整动作输出，而 a_t 是真正能在环境里执行的子动作序列。环境随后返回下一状态 s_{t+1} 和奖励 r_t 。整条轨迹里每个部分要么是 LLM 生成的、要么是环境产生的，都会被**联合优化**。

训练循环交替进行 rollout 和 update：新轨迹可用 π_θ 在线 (on-policy) 生成，也可从 replay buffer 里按 π_{old} 采样。每个训练循环有 P 个初始状态、每个生成 N 条轨迹、以批大小 E 更新、共 L 个循环，总梯度更新步数 $S = L \cdot \frac{P \cdot N}{E}$ 。

3.2.3 2.2.3 模块化的优化策略

StarPO 在统一的轨迹级抽象下支持多种优化算法。论文给了两种 token 级更新的实例：

PPO 版本（训练一个 critic 来估计 token 级价值和优势 $A_{i,t}$ ）：

$$J_{\text{PPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|\tau_i|} \sum_{t=1}^{|\tau_i|} \min \left(\frac{\pi_\theta(\tau_{i,(t)}|\tau_{i,<t})}{\pi_{\text{old}}(\tau_{i,(t)}|\tau_{i,<t})} A_{i,t}, \text{clip} \left(\frac{\pi_\theta(\tau_{i,(t)}|\tau_{i,<t})}{\pi_{\text{old}}(\tau_{i,(t)}|\tau_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) A_{i,t} \right). \quad (4)$$

GRPO 版本（不用 critic，给每条轨迹一个标量奖励 $R(\tau_i)$ ，并在轨迹内所有 token 上做归一化优势 $\hat{A}_{i,t}$ ）：

$$\hat{A}_{i,t} = \frac{R(\tau_i) - \text{mean}(\{R(\tau_1), \dots, R(\tau_G)\})}{\text{std}(\{R(\tau_1), \dots, R(\tau_G)\})}. \quad (5)$$

再代入与式 (4) 结构相同的 GRPO 目标 $J_{\text{GRPO}}(\theta)$ （把 $A_{i,t}$ 换成 $\hat{A}_{i,t}$ ，见式 6）。

怎么读这几个公式（直觉版）： G 是这批里的轨迹数； $\tau_{i,(t)}$ 是第 i 条轨迹的第 t 个 token；那个分式 $\frac{\pi_\theta}{\pi_{\text{old}}}$ 是**新旧策略的概率比**（这一步比之前更愿意/更不愿意产出这个 token）； $A_{i,t}$ （或 $\hat{A}_{i,t}$ ）是**优势**（这一步比平均水平好多少，好就拉高概率、差就压低）； $\text{clip}(\cdot, 1 - \varepsilon, 1 + \varepsilon)$ 把概率比裁剪在 $[1 - \varepsilon, 1 + \varepsilon]$ 内，**防止一步更新太猛**。两者的根本区别就在优势怎么算：PPO 靠 critic 估计，GRPO 靠”组内减均值除标准差”。

3.2.4 2.3 RAGEN 系统

RAGEN 是 StarPO 的工程落地：支持结构化 rollout、可定制奖励函数、与多轮随机环境的对接。它既是 StarPO 的执行后端，也是研究稳定性、泛化、学习动力学的平台。设计上强调**可扩展**——新环境、新奖励方案、新 rollout 策略都能即插即用。

3.3 第 3 节：实验设置

四个环境，各考一种能力（细节见后文”实验解读”）：Bandit 考”噪声反馈下的风险敏感推理”；Sokoban 考”不可逆的符号规划”；Frozen Lake 把规划和概率性转移结合；WebShop 涉及自然语言落地和网页交互。前三个符号环境刻意做得**极简、完全可控**，就是为了”看清”训练过程。

训练设置：主实验用 Qwen-2.5 Instruct 0.5B 训三个符号任务、用 3B 变体训更难的 WebShop。在 H100 GPU 上跑 100–200 个 rollout–update 迭代。每批采样 $P=8$ 个 prompt，每个 prompt 出 $N=16$

条 rollout，最多 5 轮、10 个动作。策略更新用 GRPO 或 PPO + GAE ($\gamma=1.0, \lambda=1.0$)、Adam 优化器、熵奖励 ($\beta=0.001$)、以及一个 -0.1 的格式惩罚 (response-format penalty)。

评测指标 (五个仪表盘): 每个环境用 256 个固定 prompt、温度 $T=0.5$ 、5 轮后截断。指标包括：(i) 成功率 (任务完成)；(ii) rollout 熵 (探索程度)；(iii) 组内奖励方差 (行为多样性)；(iv) 回应长度 (推理冗长度)；(v) 梯度范数 (训练稳定性)。后四个不是为了刷分，而是诊断训练健康度的”体检仪表”——这正是这篇方法论文的精髓。

4 方法图解：StarPO 框架到底在干什么

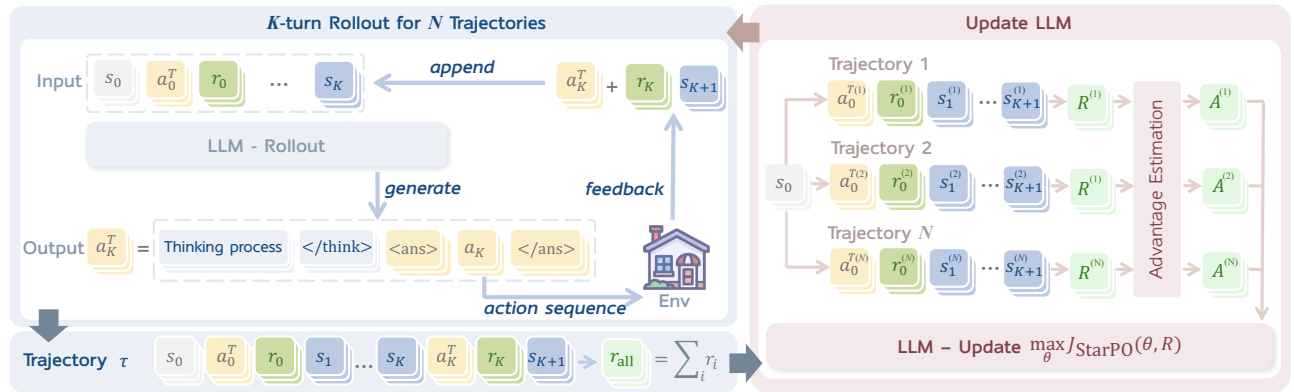


图 1: StarPO (State-Thinking-Actions-Reward Policy Optimization) 框架 (原文图 2)。LLM 生成”推理引导的动作”与环境进行多轮交互，累积轨迹级奖励，归一化后用于更新 LLM 策略。

这张图在说什么、数据怎么流动 (对照图 1，分三块看)：

左半部分——K 轮 Rollout (K-turn Rollout): 这是”模型在环境里实际跑一遍”的过程。

- 从初始状态 s_0 开始，LLM-Rollout 模块 generate (生成) 一个结构化输出 (中间那条): `<think>` 思考过程 `</think>` `<ans>` 真正动作 a_K `</ans>`。
- 这个动作序列 (action sequence) 交给 **Env** (环境，图里那个小房子) 执行，环境 feedback (反馈) 回奖励 r_K 和新状态 s_{K+1} 。
- 新的 (a_K^T, r_K, s_{K+1}) 被 append (追加) 回输入序列，进入下一轮。如此循环 K 轮。
- 最终把所有片段拼成一条完整**轨迹** τ (最下面那条带子): $s_0, a_0^T, r_0, s_1, \dots$ ，并算出整条轨迹的总奖励 $r_{all} = \sum_i r_i$ 。

右半部分——更新 LLM (Update LLM): 这是”从跑出来的轨迹里学习”的过程。

- 从同一个 s_0 出发，会有 N 条平行轨迹 (Trajectory 1、2、...、 N)，每条都有自己的奖励 $R^{(i)}$ 。
- 这 N 条轨迹一起送进 **Advantage Estimation (优势估计)** 模块，算出每条轨迹的优势 $A^{(i)}$ ——本质就是”这条比这一组的平均水平好多少”。

- 最后用这些优势去做 **LLM-Update**，优化目标 $\max_{\theta} J_{\text{StarPO}}(\theta, R)$ ，把”表现好的轨迹”对应的 token 概率拉高。

一句话总结这张图：左边”跑”出多条完整轨迹、右边按”组内相对好坏”给轨迹打优势分、再回去更新模型——这就是”轨迹级”优化区别于”单步”优化的全部精髓。

5 实验解读：数字背后说明了什么

5.1 四个测试环境（数据集/基准）

- **Bi-Arm Bandit（双臂老虎机，单轮、随机）**：在两个有”符号语义”的选项间二选一（如 Dragon vs. Phoenix）。低风险臂固定给 0.15 的奖励；高风险臂从 Bernoulli(0.25) 采样——**方差更大但期望更高**。妙处在于：低风险臂单次更常赢，但高风险臂长期更优。这就逼着模型去做风险敏感推理，把符号线索（”Dragon”）和背后的奖励统计关联起来，而不是被短期信号带偏。
- **Sokoban（推箱子，多轮、确定性）**：在格子里把箱子推到目标点。关键是**不可逆**——箱子只能推不能拉，推错就死局，逼模型提前规划。奖励：每个到位的箱子 +1，推偏 -1，完成 +10，每个动作 -0.1。
- **Frozen Lake（冰湖，多轮、随机）**：在有”滑冰格”的格子里走到目标。每个动作只有 1/3 概率成功，2/3 概率向垂直方向滑偏。奖励稀疏：成功 +1，其余全 0。
- **WebShop（网购，多轮、开放域）**：真实的多轮网购任务，要理解自然语言需求、操作半结构化界面、搜索/点击/读商品描述。引入了符号环境里没有的语言落地和动作空间挑战。

对比的基线：本文的”基线”不是别家的系统，而是把**单轮 RL 方法（PPO、GRPO）直接搬到多轮**（即 vanilla StarPO），再和稳定版 **StarPO-S** 对比；同时附录里还和更大模型（72B）、前沿模型（GPT-4o、Qwen-2.5-72B）对照，以便给小模型的表现一个参照系。

5.2 核心结果一：直接搬单轮方法会”训崩”（发现 1&2&3）

baseline StarPO 在符号任务（Bandit、Sokoban）上**先涨后崩**：早期有提升，最终塌缩。PPO 比 GRPO 稳一些（崩得更晚、峰值更高），因为它的 critic 提供更平滑的奖励估计；但在 Frozen Lake 上反而 GRPO 更稳（因为这个任务状态价值难估，会把 PPO 带乱）；WebShop 上两者都成功（强语言先验 + 高初始奖励，降低了对 critic 的依赖）。

作者对比”早期 vs. 晚期”轨迹，发现了 **Echo Trap（回声陷阱）**：早期模型对符号意义和期望奖励有**多样的推理**，晚期回应变得**重复、确定**。这说明 RL 把”局部被奖励的套路”过度放大了，反复套用记住的模板、压制了探索。原文一句关键描述：

”early-stage agent respond with diverse symbolic reasoning, but collapse into deterministic, repetitive templates after training.”

(早期智能体会用多样的符号推理来回应，但训练后塌缩成确定、重复的模板。)

怎么提前预警崩溃 (这是全文最实用的部分之一)：作者监控四个信号，并发现它们有明确的时间先后：

- **奖励标准差 (reward std) 是收敛的早期指标**：它常在奖励均值崩之前就先掉。例如 FrozenLake-PPO 在第 40 步 std 骤降，而奖励均值第 90 步才崩。
- **梯度范数尖峰 (gradient norm spike) 标志不可逆崩溃**：一旦出现尖峰 (如 Bandit 第 170 步、Sokoban 第 110 步、FrozenLake 第 90 步)，哪怕极小的参数更新也会引发剧烈损失波动，之后基本救不回来。
- **熵 (entropy) 在有效学习时应平稳衰减**：熵的急升或乱跳，往往对应推理行为已经崩坏。

发现 1&2&3 合起来的实操结论：单轮 RL 方法不能直接拿来做多轮 agent RL；模型崩溃在训练中表现为 Echo Trap；而**奖励标准差/熵的异常波动**会先于性能下降出现，**梯度范数尖峰**则标志着不可逆的崩溃点——这几个指标给了我们”在崩之前踩刹车”的早期信号。

5.3 核心结果二：StarPO-S 怎么把训练救回来 (发现 4)

基于”奖励标准差下降常常预示崩溃”这个洞见，作者提出一个反直觉但有效的假设：是不是应该把训练资源更多投在那些”行为更不确定、奖励方差更高”的任务实例上？——这正是**主动学习 (Active Learning)**的思想：不确定的样本信息量最大。

为此定义**轨迹级结果不确定性** U (对初始状态 s_0)：

$$U(\pi_\theta, \mathcal{M}, s_0) = \text{Std}_{\tau \sim \pi_\theta(\cdot | s_0)} [R(\tau)]. \quad (7)$$

拆解 (式 7)：对同一个初始状态 s_0 反复 rollout 多条轨迹，看它们最终奖励 $R(\tau)$ 的**标准差**。std 大 = 这道题模型时是对时错、最值得学；std 小 = 要么稳过、要么稳挂，信息量低。训练时按这个 std 给 prompt 排序，**只保留方差最高的 top $p\%$** 。

StarPO-S 的三件套：

1. **基于不确定性的轨迹过滤**：扔掉低方差 rollout。PPO 上效果显著——保留 75% 能把 FrozenLake 的稳定期从 100 步延到 140 步，保留 50% 干脆完全不崩。默认采用激进的**保留 25%** (但作者也提醒这未必对所有场景最优)。过滤还顺带**提速** (少算了没信息量的样本)。
2. **KL 项移除 (KL Term Removal)**：借鉴 DAPO，去掉 KL 惩罚项，给优化更大自由度。
3. **Clip-Higher (非对称裁剪, Asymmetric Clipping)**：也来自 DAPO，把裁剪上下界解耦，鼓励对低概率但有潜力的 token 多一点探索。

发现 4：在高方差 **prompt** 上训练，能延迟甚至消除多轮 RL 的崩溃。StarPO-S 通过丢弃低信息量 rollout，既**提升性能**又**减少更新步数**（PPO 上尤其明显），与主动学习原理一致。整体对比（原文图 6）显示，StarPO-S 在全部四个任务上都**延迟了崩溃、并取得更高成功率**。

5.4 核心结果三：什么样的 rollout 才是”好教材”（发现 5）

作者在 vanilla Sokoban 上训练、在变体任务（SokobanNewVocab、LargeSokoban、FrozenLake）上评测，研究三个 rollout 维度。下面把关键数字重排成小表。

表 1：任务多样性对泛化的影响（%）（原文表 1）。固定批大小下，”每个 prompt 的回应数”越少 = 不同 prompt 越多 = 多样性越高。结论：每个 prompt 出 **4** 条回应时泛化最好。

每个 prompt 的回应数	SingleSokoban	SokobanNewVocab	FrozenLake
32	21.09	20.22	17.97
16	20.31	21.48	19.53
8	20.31	19.53	17.19
4	20.70	25.39	21.48
2	19.92	25.00	12.50
1	19.53	22.27	12.50

解读表 1：多样性高（不同初始状态多）泛化更好，但前提是**每个 prompt 仍要有多条 rollout**（否则没法”对比不同结果”）。所以”4 条/prompt”是甜区：既保证了 prompt 多样，又保留了组内对比；而”1 条/prompt”两端都垮（FrozenLake 只有 12.50%）。

表 2：不同”每轮动作预算”下各环境的表现（%）（原文表 2）。**5–6** 个动作/轮最好，给规划留足空间又不至于太乱。

每轮最大动作数	Sokoban	SokobanNewVocab	LargeSokoban	FrozenLake
1	12.11	13.67	1.17	11.72
2	16.41	21.09	3.52	18.36
3	19.53	19.53	1.95	20.88
4	26.95	26.95	5.08	20.70
5	28.13	25.78	6.25	21.09
6	33.59	31.64	6.64	18.36
7	22.27	28.52	3.91	19.53

解读表 2：动作预算太小（1-2）规划空间不够；太大（7）则注入噪声、稀释奖励反馈。**适中（5-6）**最佳——这是一个明确的”交互粒度”调参建议。

第三个维度——rollout 频率 (Online- k): 用一组 rollout 连续更新 k 次策略, k 越小越”新鲜”。Online-1 即完全在线 (每次更新都用新鲜 rollout)。结果 (原文图 7): **越新鲜 (Online-1) 收敛越快、泛化越好**; 用陈旧数据 (Online-5/10) 则变差。

发现 5: 任务多样性、动作预算、rollout 频率共同决定 rollout 质量。**多样的初始状态**带来更好的策略对比和泛化; **适中的动作预算**给足规划空间又不引入噪声; **够新鲜的 rollout** 让优化目标和当前策略保持对齐。

5.5 核心结果四：推理能帮泛化，但在多轮里会”自己消失” (发现 6)

这是全文最发人深省的一节。作者设计了一个受控泛化测试：在 Bandit 上用 [Teacher, Engineer] 这对臂训练、在 [Librarian, Trader] 上评测 (保持”高风险高回报”的直觉对应); 再做一个 **BanditRev** (把符号-奖励关系**反转**, 让推理变得反直觉、更难)。

表 3: 有/无推理时的泛化表现 (%), 均在 StarPO-S 下 (原文表 3)。关掉推理 (NoThink) 在**单轮 Bandit** 上显著掉点, 但在**多轮 Sokoban** 上影响很小甚至混杂。

	在 Bandit 上训练			在 Sokoban 上训练	
	Bandit	Bandit-Rev	FrozenLake	LargeSokoban / Sokoban	SokobanNewVocab
StarPO-S	100.00	67.58	19.92	2.34 / 21.48	18.75
NoThink	81.25	56.25	19.53	2.73 / 20.73	26.17

解读表 3: 在**单轮 Bandit**, 带推理痕迹的模型泛化明显更好 (100% vs. 81.25%), 即便在反直觉的 BanditRev 也是 (67.58% vs. 56.25%) ——说明推理帮模型**内化了符号-奖励关联**, 而非死记硬背。但在**多轮 Sokoban**, 去掉推理 (NoThink) 反而打平甚至更好 (SokobanNewVocab 上 26.17% > 18.75%)。

为什么多轮里推理会失效? 看推理长度的变化:

表 4: 不同训练步数下的推理长度 (<think> 块的 token 数) (原文表 4)。token 长度普遍随训练**下降**, 而”上下文更难”的 ReverseBandit 需要的推理更多、衰减更慢。

Step	Bandit				Sokoban	
	Original	NoThink	Reverse	Reverse-NoThink	Original	NoThink
0	66.0	12.7	68.8	12.7	307.1	68.6
100	25.6	12.4	33.7	13.0	104.6	55.3
200	17.6	12.4	30.7	13.0	89.5	60.0

解读表 4: 推理痕迹随训练**持续缩短** (如 Sokoban Original 从 307 token 掉到 89), 说明模型在

主动抑制自己的思考过程。有意思的是，在”语义错位、必须靠推理”的 BanditRev 里，推理保持得更长——印证了”上下文越难、推理越被保留”。

根因（作者的推断）：多轮任务的奖励稀疏、延迟，常常分不清”是靠连贯推理成功的”还是”瞎试碰对的”。附录里就有例子：模型生成了不连贯/幻觉式的推理，却照样拿高奖励。这就引出一个核心难题：当奖励本身无法反映推理质量时，怎么才能稳定地强化”有用的推理”？

发现 6：符号推理能在简单的单轮任务里、弱监督下涌现，但在多轮环境里若奖励不显式鼓励可解释的中间推理步骤，推理就会逐渐衰减。一个初步对策是用基于格式的惩罚（输出缺少有效的 `<think>--<answer>` 结构就扣分）；作者呼吁未来探索更细粒度的奖励设计（如奖励”部分正确”）来可靠地强化长时程决策中的推理。

5.6 第 5&6 节：相关工作与结论/局限

相关工作一句话带过：本文站在两条线的交叉口——一条是”用 RL 提升 LLM 推理”（PPO、GRPO、DAPO、STaR、PRM、MCTS 等），一条是”agent 框架”（从 ReAct、Reflexion 等早期反应式规划，到多智能体协作、具身交互；基准如 Sokoban、FrozenLake、WebShop）。本文要把”基于 RL 的推理”和”结构化 agent 训练”在符号与语言任务上统一起来。

结论与局限：RAGEN 是一个在多轮、随机环境里用 RL 训练语言智能体的通用系统；它揭示了 agent 训练独有的新挑战——**梯度崩溃（gradient collapse）、rollout 漂移（rollout drift）、推理退化（reasoning degradation）**，并给出稳定训练的关键设计原则（rollout 过滤、梯度塑形、感知推理的奖励监督）。作者坦诚的**局限**：聚焦相对小规模任务、省略了 replay buffer 等成熟 RL 实践、未涉及多模态任务——留作未来工作。

6 核心 takeaways: 给 AI 应用开发者的启示

1. **多轮 agent RL 不是把单轮方法”加几轮”那么简单。**直接把 PPO/GRPO 搬过来，多半会先涨后崩。如果你打算自己训 agent，要预期到不稳定，并**提前准备稳定化手段**，而不是等崩了再救。
2. **把”奖励标准差、熵、梯度范数”当训练体检仪。**它们能在成功率掉之前给出预警：奖励 *std* / 熵异常波动是早期信号，梯度范数尖峰是”不可逆崩溃”的红线。在你的训练脚本里把这几个指标打到监控面板上，比只盯着 loss/成功率有用得多。
3. **”难样本优先”是个高性价比的稳定技巧。**StarPO-S 的核心——只在”模型时与时错（高奖励方差）”的实例上重点训练——既稳又省。这对应主动学习直觉，且实现简单（按 rollout 奖励 *std* 排序、保留 top *p*%）。
4. **rollout 的”配方”很关键，有明确的调参甜区：**初始状态要多样、但每个 prompt 仍要有多条 rollout 做对比（甜区约 4 条/prompt）；每轮动作预算**适中**（5-6 个）；rollout 要**新鲜**（尽量在线、Online-1）。这些都是可以直接照搬的工程默认值。

5. ”让模型加 `<think>` 标签”并不等于”模型真的会推理”。如果奖励只看最终成败、不看推理质量，模型会**偷懒退化**（推理越来越短）甚至**编幻觉推理却照样拿高分**。想让推理真正发挥作用，必须设计**感知推理、细粒度**的奖励（哪怕只是格式惩罚这种轻量手段，也比什么都不做强）。
6. **方法论价值大于”刷榜”价值**。这篇刻意用 0.5B 小模型 + 玩具环境，不是为了 SOTA，而是为了**把训练动力学看清楚**。对开发者的启示是：在投入大模型大算力之前，先在**极简可控环境**里验证你的训练配方是否稳定——这能省下大量试错成本。

参考文献（仅列最关键几条，英文原样）

1. J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017. <https://arxiv.org/abs/1707.06347>.
2. Z. Shao, P. Wang, Q. Zhu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024. <https://arxiv.org/abs/2402.03300>. （GRPO 出处）
3. DeepSeek-AI, D. Guo, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025. <https://arxiv.org/abs/2501.12948>.
4. Q. Yu, Z. Zhang, R. Zhu, et al. DAPO: An open-source LLM reinforcement learning system at scale, 2025. <https://arxiv.org/abs/2503.14476>. （KL 移除 + Clip-Higher 出处）
5. I. Shumailov, Z. Shumaylov, Y. Zhao, et al. AI models collapse when trained on recursively generated data. *Nature*, 631(8022):755–759, 2024. <https://doi.org/10.1038/s41586-024-07566-y>. （与 Echo Trap 相关）
6. S. Yao, H. Chen, J. Yang, and K. Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. *NeurIPS*, 2022.
7. Y. Zhou, A. Zanette, J. Pan, S. Levine, and A. Kumar. ArCHer: Training language model agents via hierarchical multi-turn RL, 2024. <https://arxiv.org/abs/2402.19446>.